

Chapter-4

Requirement Engineering

Requirements Engineering

- ✓ The process of establishing the services that the customer requires from a system **and** the constraints under which it operates and is developed.
- ✓ The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process.

What is a Requirement?

- ✓ It may range from a high-level abstract statement of a service OR of a system constraint to a detailed mathematical functional specification.
- ✓ This is inevitable as requirements may serve a dual function
 - ✓ May be the basis for a bid for a contract - therefore must be open to interpretation;
 - ✓ May be the basis for the contract itself - therefore must be defined in detail;
 - ✓ Both these statements may be called requirements.

Requirements Abstraction (Davis)

- “If a company wishes to let a contract for a large software development project, it must define its needs in a sufficiently abstract way that a solution is not pre-defined.
- The requirements must be written so that several contractors can bid for the contract, offering, perhaps, different ways of meeting the client organization’s needs.

Once a contract has been awarded, the contractor must write a system definition for the client in more detail so that the client understands and can validate what the software will do.

Both of these documents may be called the requirements document for the system.”

Types of Requirements

- ✓ User requirements
 - ✓ Statements in natural language plus diagrams of the services the system provides and its operational constraints.
 - ✓ Written for customers.
- ✓ System requirements
 - ✓ A structured document setting out detailed descriptions of the system's functions, services and operational constraints.
 - ✓ Defines what should be implemented so may be part of a contract between client and contractor.
 - ✓ Whom do you think these are written for?
 - ✓ These are higher level than functional and non-functional requirements, which these may subsume.

User and System Requirements

User requirement definition

1. The MHC-PMS shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1 On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2 The system shall automatically generate the report for printing after 17.30 on the last working day of the month.
- 1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4 If drugs are available in different dose units (e.g. 10mg, 20 mg, etc.) separate reports shall be created for each dose unit.
- 1.5 Access to all cost reports shall be restricted to authorized users listed on a management access control list.

Functional and Non-functional requirements

- ✓ **Functional requirements**
 - ✓ Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
 - ✓ May state what the system should not do.
- ✓ **Non-functional requirements**
 - ✓ Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
 - ✓ Often apply to the system as a whole rather than individual features or services.
- ✓ **Domain requirements**
 - ✓ Constraints on the system from the domain of operation

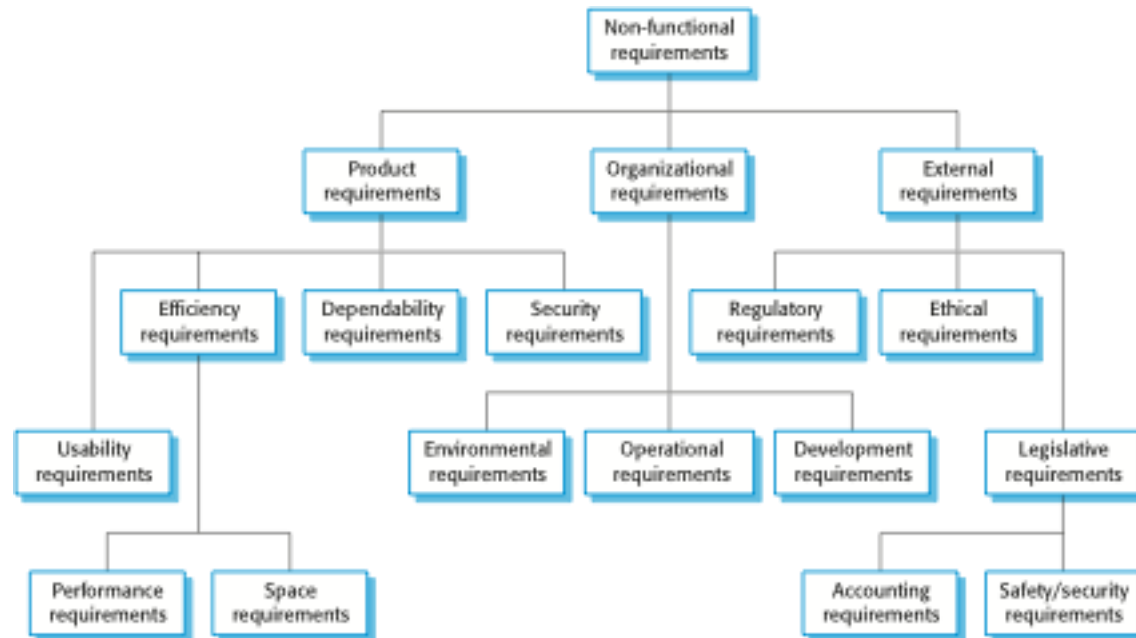
Functional Requirements

- ✓ Describe functionality or system services.
- ✓ Depend on the type of software, expected users and the type of system where the software is used.
- ✓ Functional user requirements may be high-level statements of what the system should do.
- ✓ Functional system requirements should describe the system services in detail.
- ✓ Essentially, these are the ‘whats’ of the system that we often refer to. These are not ‘all that there is,’ but these should describe the overall functionality of the system.

Non-functional Requirements

- ✓ These define system properties and constraints e.g. reliability, response time, maintainability, scalability, portability, and storage requirements.
- ✓ Constraints are I/O device capability, system representations, etc.
- ✓ Process requirements may also be specified mandating a particular IDE, programming language or development method.
- ✓ (Often internal to an organization or required for fit / compatibility with other comparable systems.)
- ✓ Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless

Types of Nonfunctional Requirements



Non-functional

- ✓ Non-functional requirements may affect the overall architecture of a system rather than the individual components.
 - ✓ For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
- ✓ A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
 - ✓ It may also generate requirements that restrict existing requirements

Non-functional Requirements Implementation

- ✓ Non-functional requirements may affect the overall architecture of a system rather than the individual components.
 - ✓ For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
- ✓ A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
 - ✓ It may also generate requirements that restrict existing requirements.

Metrics for specifying nonfunctional requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure (MTTF) Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure (MTTR) Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Domain requirements problems

✓ Understandability

- ✓ Requirements are expressed in the language of the application domain;
 - ✓ *Application written for mortgage banking people need to express functionality in terms of home loans, mortgage balances, escrow, investor accounting, foreclosure, etc.*
- ✓ This is often not understood by software engineers developing the system.

✓ Implicitness

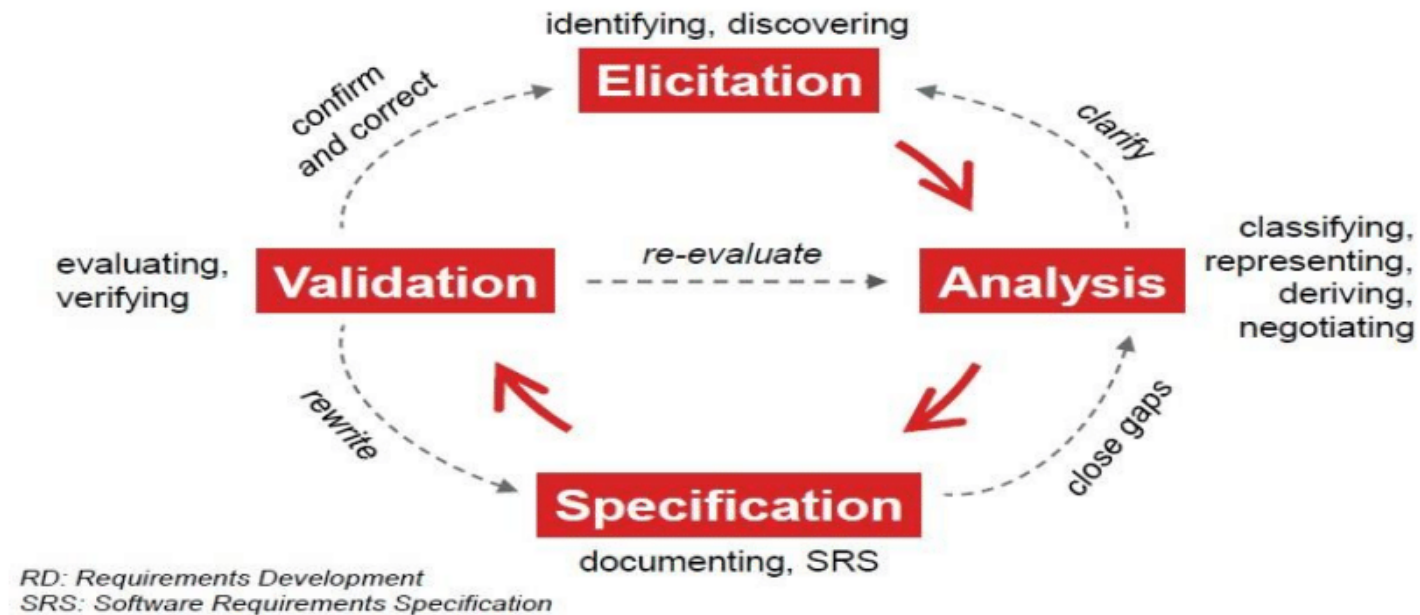
- ✓ Domain specialists understand the area so well that they do not think of making the domain requirements explicit.
- ✓ And this is often a major problem in communications!!!

Requirements engineering processes

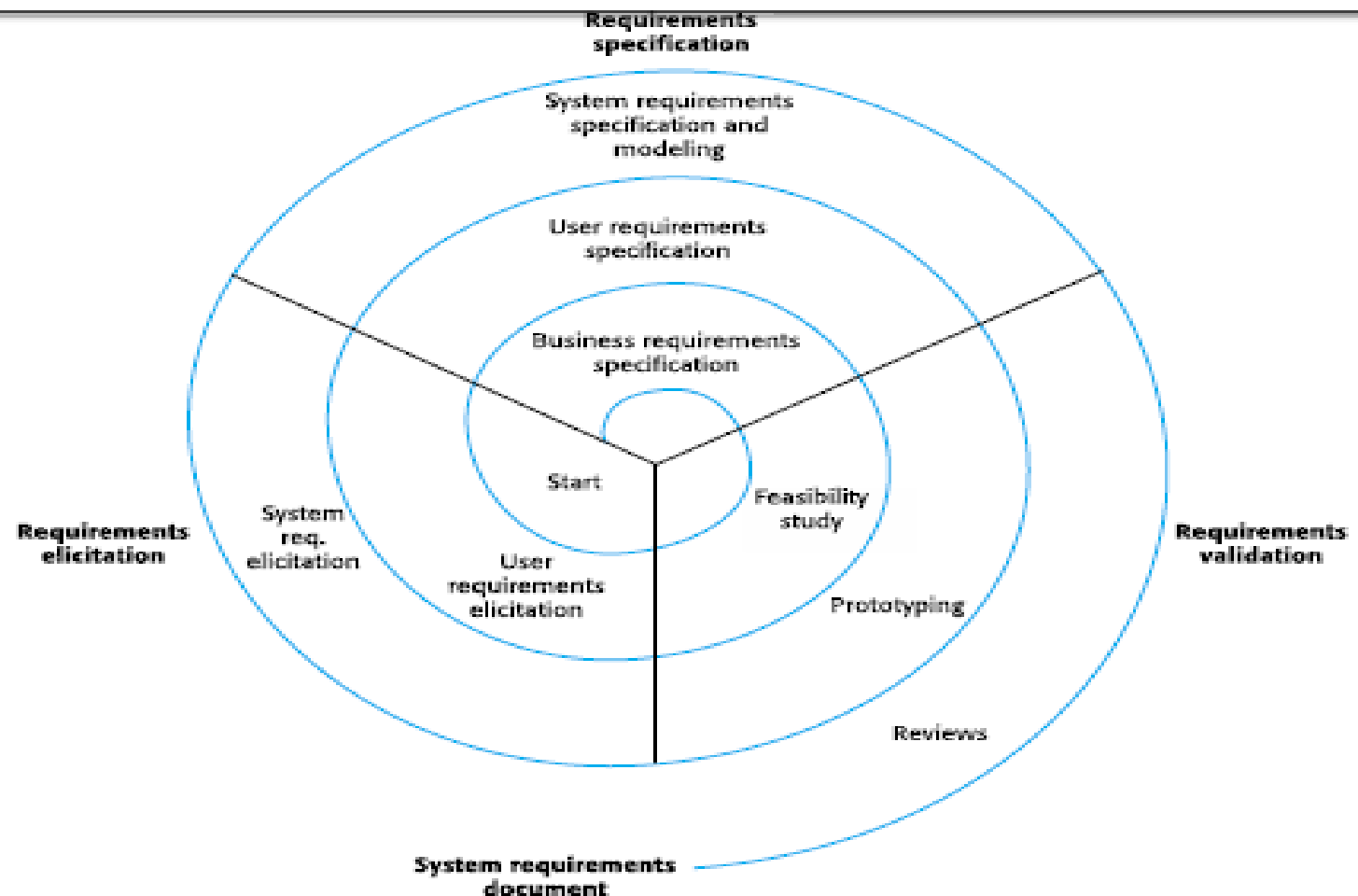
- The processes used for RE vary widely depending on the application domain, the people involved and the organisation developing the requirements.
- However, there are a number of generic activities common to all processes
 - Requirements elicitation;
 - Requirements analysis;
 - Requirements validation;
 - Requirements management.

In practice, RE is an iterative activity in which these processes are interleaved.

RD Process Framework



A spiral view of the requirements engineering process



Requirements elicitation and analysis

- Sometimes called requirements elicitation or requirements discovery.
- Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints.
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called stakeholders.

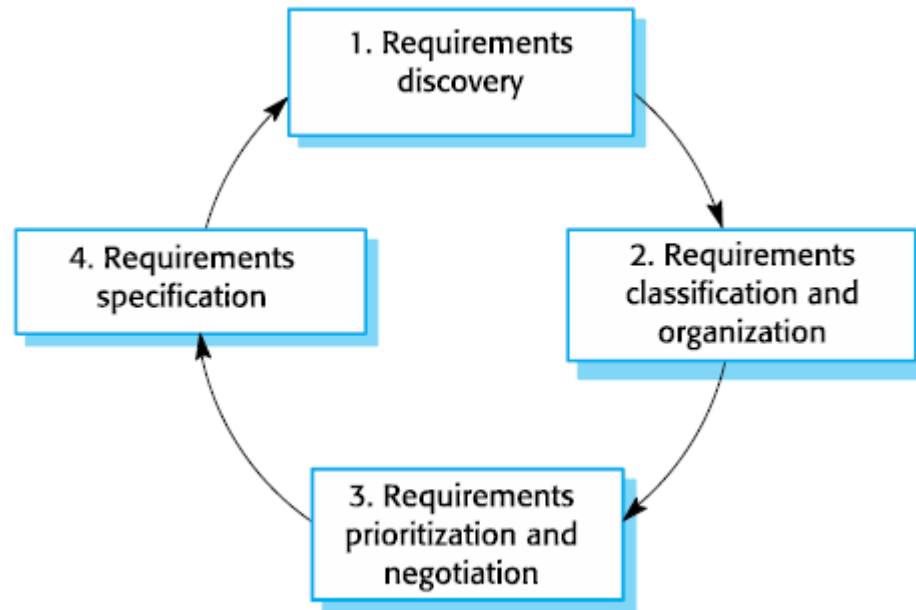
Requirements elicitation

- Software engineers work with a range of system stakeholders to find out about the application domain, the services that the system should provide, the required system performance, hardware constraints, other systems, etc.
- Stages include:
 - Requirements discovery,
 - Requirements classification and organization,
 - Requirements prioritization and negotiation,
 - Requirements specification.

Problems of requirements elicitation

- Stakeholders don't know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process.
- New stakeholders may emerge and the business environment may change.

The requirements elicitation and analysis process



Process activities

- ✧ Requirements discovery
 - Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- ✧ Requirements classification and organisation
 - Groups related requirements and organises them into coherent clusters.
- ✧ Prioritisation and negotiation
 - Prioritising requirements and resolving requirements conflicts.
- ✧ Requirements specification
 - Requirements are documented and input into the next round of the spiral.

Requirements specification

- The process of writing down the user and system requirements in a requirements document.
- User requirements have to be understandable by end users and customers who do not have a technical background.
- System requirements are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development ,It is therefore important that these are as complete as possible.

Ways of writing a system requirements specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Requirements and design

- ✧ In principle, requirements should state what the system should do and the design should describe how it does this.
- ✧ In practice, requirements and design are inseparable
 - ■ A system architecture may be designed to structure the requirements;
 - ■ The system may inter-operate with other systems that generate design requirements;
 - ■ The use of a specific architecture to satisfy non-functional requirements may be a domain requirement.
 - ■ This may be the consequence of a regulatory requirement.

Natural language specification

- Requirements are written as natural language sentences supplemented by diagrams and tables.
- Used for writing requirements because it is expressive, intuitive and universal. This means that the requirements can be understood by users and customers.

Guidelines for writing requirements

- Invent a standard format and use it for all requirements.
- Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary.

Problems with natural language

- Lack of clarity
- Precision is difficult without making the document difficult to read.
- Requirements confusion
- Functional and non-functional requirements tend to be mixed-up.
- Requirements amalgamation
- Several different requirements may be expressed together

Example requirements for the insulin pump software system

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. *(Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)*

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. *(A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)*

Structured specifications

- An approach to writing requirements where the freedom of the requirements writer is limited and requirements are written in a standard way.
- This works well for some types of requirements e.g. requirements for embedded control system but is sometimes too rigid for writing business system requirements.

Form-based specifications

- Definition of the function or entity.
- Description of inputs and where they come from.
- Description of outputs and where they go to.
- Information about the information needed for the computation and other entities used.
- Description of the action to be taken.
- Pre and post conditions (if appropriate).
- The side effects (if any) of the function

A structured specification of a requirement for an insulin pump

Insulin Pump/Control Software/SRS/3.3.2

Function Compute insulin dose: safe sugar level.

Description

Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.

Inputs Current sugar reading (r2); the previous two readings (r0 and r1).

Source Current sugar reading from sensor. Other readings from memory.

Outputs CompDose—the dose in insulin to be delivered.

Destination Main control loop.

A structured specification of a requirement for an insulin pump

Action

CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.

Requirements

Two previous readings so that the rate of change of sugar level can be computed.

Pre-condition

The insulin reservoir contains at least the maximum allowed single dose of insulin.

Post-condition r_0 is replaced by r_1 then r_1 is replaced by r_2 .

Side effects None.

Tabular specification

- Used to supplement natural language.
- Particularly useful when you have to define a number of possible alternative courses of action.
- For example, the insulin pump systems bases its computations on the rate of change of blood sugar level and the tabular specification explains how to calculate the insulin requirement for different scenarios.

Tabular specification of computation for an insulin pump

Condition	Action
Sugar level falling ($r_2 < r_1$)	CompDose = 0
Sugar level stable ($r_2 = r_1$)	CompDose = 0
Sugar level increasing and rate of increase decreasing ($(r_2 - r_1) < (r_1 - r_0)$)	CompDose = 0
Sugar level increasing and rate of increase stable or increasing ($(r_2 - r_1) \geq (r_1 - r_0)$)	CompDose = round $((r_2 - r_1)/4)$ If rounded result = 0 then CompDose = MinimumDose

Requirements validation

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
- Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking

- Validity. Does the system provide the functions which best support the customer's needs?
- Consistency. Are there any requirements conflicts?
- Completeness. Are all functions required by the customer included?
- Realism. Can the requirements be implemented given available budget and technology
- Verifiability. Can the requirements be checked?

Requirements validation techniques

- Requirements reviews
- Systematic manual analysis of the requirements.
- Prototyping
- Using an executable model of the system to check requirements.
- Test-case generation
- Developing tests for requirements to check testability.

Requirements reviews

- Regular reviews should be held while the requirements definition is being formulated.
- Both client and contractor staff should be involved in reviews.
- Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Review checks

- **Verifiability**

Is the requirement realistically testable?

- **Comprehensibility**

Is the requirement properly understood?

- **Traceability**

Is the origin of the requirement clearly stated?

- **Adaptability**

Can the requirement be changed without a large impact on other requirements?

Requirements change

- The business and technical environment of the system always changes after installation.
- New hardware may be introduced, it may be necessary to
- interface the system with other systems, business priorities may change
- The people who pay for a system and the users of that system are rarely the same people.

- System customers impose requirements because of organizational and budgetary constraints. These may conflict with end-user requirements and, after delivery, new features may have to be added for user support if the system is to meet its goals.
- Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory.
- The final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.

Requirements management

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- New requirements emerge as a system is being developed and after it has gone into use.
- You need to keep track of individual requirements and maintain links between dependent requirements so that you can assess the impact of requirements changes.
- You need to establish a formal process for making change proposals and linking these to system requirements.

Requirements management planning

- Establishes the level of requirements management detail that is required.
- Requirements management decisions:
 - *Requirements identification* Each requirement must be uniquely identified so that it can be cross-referenced with other requirements.
 - *A change management process* This is the set of activities that assess the impact and cost of changes. I discuss this process in more detail in the following section.
 - *Traceability policies* These policies define the relationships between each requirement and between the requirements and the system design that should be recorded.
 - *Tool support* Tools that may be used range from specialist requirements management systems to spreadsheets and simple database systems.

Requirements change management

- Deciding if a requirements change should be accepted
 - *Problem analysis and change specification*
- During this stage, the problem or the change proposal is analyzed to check that it is valid. This analysis is fed back to the change requestor who may respond with a more specific requirements change proposal, or decide to withdraw the request.
 - *Change analysis and costing*
- The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements.
- Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.
- Change implementation
 - The requirements document and, where necessary, the system design and implementation, are modified. Ideally, the document should be organized so that changes can be easily implemented.

Requirements change management

